```
# ============================================================
# PR4: Knowledge Representation and Expert System
# Forward Chaining and Backward Chaining
# ============================================================



# ------------------------------------------------------------
# Knowledge Base
# ------------------------------------------------------------

facts = {
   "fever",
   "cough",
   "body_pain"
}



# Rules are represented as:
# IF conditions THEN conclusion

rules = [
  {
     "conditions": ["fever", "cough"],
     "conclusion": "flu"
  },


  {
     "conditions": ["flu", "body_pain"],
     "conclusion": "viral_infection"
  },
```

```
  {
    "conditions": ["viral_infection"],
    "conclusion": "take_rest"
  },

  {
    "conditions": ["take_rest"],
    "conclusion": "recovery"
  }
]



# ============================================================
# FORWARD CHAINING
# ============================================================

def forward_chaining(facts, rules):

  known_facts = set(facts)

  print("\n========== FORWARD CHAINING ==========")

  changed = True

  while changed:

    changed = False

    for rule in rules:

      conditions = set(rule["conditions"])
```

```
        conclusion = rule["conclusion"]

        # Check whether all conditions are satisfied
        if conditions.issubset(known_facts):

          if conclusion not in known_facts:

            print(
              "Rule Fired:",
              " AND ".join(rule["conditions"]),
              "->",
              conclusion
            )

            known_facts.add(conclusion)
            changed = True

  return known_facts


# ============================================================
# BACKWARD CHAINING
# ============================================================

def backward_chaining(goal, facts, rules, visited=None):

  if visited is None:
    visited = set()

  # Goal already known
  if goal in facts:
```

```
    return True

# Avoid infinite loops
if goal in visited:
    return False

visited.add(goal)

# Find rules that can produce the goal
for rule in rules:

    if rule["conclusion"] == goal:

        print(
            "Trying rule:",
            " AND ".join(rule["conditions"]),
            "->",
            goal
        )

        all_conditions_true = True

        # Try to prove every condition
        for condition in rule["conditions"]:

            if not backward_chaining(
                condition,
                facts,
                rules,
                visited
            ):
```

```
            all_conditions_true = False
            break

        if all_conditions_true:
            return True

    return False


# ============================================================
# Execute Forward Chaining
# ============================================================

derived_facts = forward_chaining(facts, rules)

print("\nFacts after Forward Chaining:")

for fact in sorted(derived_facts):
    print("-", fact)


# ============================================================
# Execute Backward Chaining
# ============================================================

goal = "recovery"

print("\n========== BACKWARD CHAINING ==========")

result = backward_chaining(
    goal,
```

```
    facts,
    rules
)



# ============================================================
# Display Final Decision
# ============================================================

print("\n========== FINAL DECISION ==========")

print("Goal:", goal)

if result:
    print("Result: Goal can be proved.")
else:
    print("Result: Goal cannot be proved.")
```